Quick Work

Page No.:

Date:

M T W T F S S

Array: [45, 22, 67, 19, 50, 89, 33, 72, 40, 55]

WAP to

1. Construct an AVL tree by inserting the given array elements sequentially.
   - Ensure tree remains balanced after each insertion using appropriate rotations (LL, RR, LR, RL).
2. Display the rotations performed
3. Implement deletion of a given key from the AVL tree
   Rebalance the tree after deletion if required.

```
#include <stdio.h>
#include <stdlib.h>

typedef ~~Node*~~ struct Node {
    int data;
    int height;
    Node* left;
    Node* right;
} node;

void CreateNode(int data){
~~Node* Node~~
newNode = (node*) malloc(sizeof(node*));
newNode->data = data;
newNode->height = 0;
newNode->left = NULL;
newNode->right = NULL;
```

Quick Work

Page No.:

Date:

M T W T F S S

```
}
}
int GetLeftHeight (root*){
int leftH = 0;
while (root != NULL){
leftH = leftH + 1;
GetLeftHeight (root->left);
}
return leftH;
}
Int GetRightHeight (root*)
int rightH = 0;
while (root != NULL){
rightH = rightH + 1;
GetRightHeight (root->right);
}
return rightH;
}
Int GetHeight (root*){
int height = max (GetLeftHeight (root*), GetRightHeight (root*)) + 1;
return height;
}
Int GetBalance (root*){
int balance = GetLeftHeight (root) - GetRightHeight (root);
return balance;
}
```

Quick Work

Page No.:

Date:

M T W T F S S

~~void Insertion (int data) {~~

```
Node* LeftRotation (root*) {
    Node* x = root -> right;
    Node* T2 = x -> left;
    Node* y = root;

    x -> left = y;
    x -> right = T2;   y -> left = NULL; y -> right = NULL;
    return x;
}

Node* RightRotation (root*) {
    Node* x = root -> left;
    Node* T2 = x -> right;
    Node* y = root;

    x -> left = T2;
    x -> right = y;
    y -> left = NULL;
    y -> right = NULL;
    return y;
}
```

28/04/25

```
Node* insertion (Node *root, int data) {
    if (root == NULL) {
        root = CreateNode(data);
        return (root);
    }
    if (data > root -> data) {
        root -> right = insertion (root -> right, data);
    } else if (data < root -> data) {
        root -> left = insertion (root -> left, data);
    }
}
```